TECHNICAL DOCUMENTATION

TrustNav

Live trust monitoring for satellite positions: detection, explanation and response to GNSS jamming, spoofing and forgery, for vehicle and drone fleets.

99.3 %	96.1 %	0.44 %	8 km → 29 m
precision	recall	false alarms	worst lie, before and after the response layer

What it is. Every second, for each vehicle, TrustNav scores whether its satellite position can be believed (0 to 100), explains why in plain words, and refuses positions it does not believe, tracking the vehicle by its odometer instead. Three kinds of evidence are fused: physical rules, an anomaly model trained on normal driving only, and Galileo's own digital signatures (OSNMA).

Where it is. Live demo at trustnav.vercel.app. Source, tests and this document at github.com/domail2karandeshmukh-cyber/trustnav. Our attacks are simulated in software and the real ones are recordings made by others; nothing is transmitted.

On real attacks. Replayed on public recordings from Jammertest 2024: the first version missed a phone captured by a real spoofer for 21 min 49 s; a sixteenth rule on receiver time now refuses every captured fix from the first second.

Group 2, MSc Cybersecurity and Data Science, ESAIP Angers
Karan Deshmukh, Sudheer Reddy Jogi, Niranjan Satheesh Thandassery, Venkata Pavan Kumar Sanam
Tutor: Sofiane Hamrioui · Engine 2.2.0 · 11 November 2026

Contents

1 Summary

2 Objectives: what TrustNav does in real life

3 How it works, step by step

4 The console, in plain words

5 Code guide: every file and what it does

6 Results

7 How TrustNav differs from what exists today

8 Roadmap: prevention and improvements

9 Safety, legal and ethics

10 Team and contributions

A Appendix: running the project

Sections 1 and 4 are written for a reader who has never seen the project. Sections 3, 5 and 6 are the technical detail. Section 7 answers "why is this different from what exists?" and section 8 "does it prevent attacks, and how does it improve?".

1 Summary

What. TrustNav is live trust monitoring for satellite positions. For every vehicle and drone in a fleet, once per second, it answers *"can this position be believed?"* with a score from 0 to 100, says *why* in plain words, and, since version 2.1, *acts*: a position it does not believe is refused and replaced by an estimate from the odometer along the planned route, with a recommended action for the operator.

Why. Jamming (drowning the satellite signal) and spoofing (faking it) have become everyday events in Europe. The equipment costs 30 to 300 EUR. Ordinary fleet software shows whatever position the receiver reports and never asks whether it is true. Existing detection products either live inside one receiver and output a flag with no explanation, or are fixed monitoring stations for sites; none gives a fleet operator a per-vehicle, explained, actionable trust score (see COMPARISON.md (section 7)).

How. Three kinds of evidence, fused:

1. **Rules.** Sixteen checks (a van cannot move 8 km in a second; a signal cannot be stronger and more uniform than real satellites; the receiver's time cannot jump against the clock of the device it is plugged into) with named thresholds, so every alert can be verified by hand.
2. **Model.** An Isolation Forest and a per-feature tail detector trained on *normal* driving only, which notice the slow, careful attacks the rules miss (a van 12 m off its road).
3. **Galileo OSNMA.** The satellites' own digital signatures (operational since July 2025): a failed signature is strong evidence of forgery.

Rules and model are combined by noisy-OR (either witness can raise the alarm); trust = 100 minus the combined risk. An incident opens after three seconds below 50. The response layer refuses fixes below 50, dead-reckons along the route and releases only when a clean fix rejoins the estimate.

Results (held-out routes, synthetic data): precision 99.3 %, recall 96.1 %, F1 0.977, false alarm rate 0.44 %, 7 of 7 attack types caught; the stealth drift (OSNMA still valid) is caught after 42 s, where a fixed 150 m rule would need about 3.5 minutes. Response layer: worst position error seen by an ordinary screen 8.0 km, after the response layer 29 m; jamming blank screen 100 s without, 0 s with.

Built. Python engine (36 features, 16 rules, scikit-learn, 42 automated tests), Supabase cloud (Postgres, row-level security, realtime, token-authenticated ingest), Next.js console with a 3D street map of Angers, replay of ten recorded scenarios, a narrated attack story, a space-time view, and a live demo at https://trustnav.vercel.app. A receiver adapter for u-blox NMEA/UBX output is written and tested, ready for the first real antenna.

Real drive, no attack. On a public 27.6-minute u-blox ZED-F9K drive recording the rules raise no false alarm, the simulator-trained model must be re-calibrated on the receiver (it alerts on 89 % of the seconds before, on none of the last 8.5 minutes after), and overlaid teleport, drift and jamming are then caught in 0, 35 and 1 s.

Real attacks. Public recordings from Jammertest 2024 in Norway (VAARA dataset, University of Vaasa) were replayed unchanged. A phone was captured by a real spoofer for 26.5 minutes and ended 10.2 km from the truth: the version first published alerted only after 21 min 49 s and refused 5 of 909 captured fixes, because nothing in the spoofed stream was physically impossible. What the spoofer could not hide was time, so a sixteenth rule checks the receiver's time against the host clock: the alert now comes in the same second, all 909 captured fixes are refused, and there is no false alert in the clean periods. Real jamming of a u-blox receiver is alerted in 15 of 20 exposures (1 of 20 by the rules alone).

Limits. The detection table comes from one simulator, so those numbers are an upper bound; the real attacks are two days of one campaign. Nothing is transmitted: our attacks exist only in software and the real ones are recordings made by others. It detects and responds; it cannot remove a jammer from the air.

Next. Plug in a real receiver (about 60 to 250 EUR), record hours of normal drives around Angers, re-calibrate, then run the raw radio recordings of real attacks (Jammertest 2023, TEXBAT) through a software receiver. See ROADMAP.md (section 8).

Team. Group 2, MSc Cybersecurity & Data Science, ESAIP Angers: Karan Deshmukh, Sudheer Reddy Jogi, Niranjan Satheesh Thandassery, Venkata Pavan Kumar Sanam. Tutor: Sofiane Hamrioui.

2 Objectives: what TrustNav does in real life

The problem

Almost everything that moves today trusts its satellite position (GPS, Galileo, GLONASS, BeiDou, together called GNSS) without question: delivery vans, trucks, taxis, ambulances, trams, ships, agricultural machines and drones.

That trust is easy to break:

- **Jamming**: a small radio transmitter drowns the weak satellite signal. The receiver loses its position. Cheap jammers (about €30) are sold illegally online and are used by drivers who want to hide from fleet trackers, and by thieves.
- **Spoofing**: a transmitter imitates the satellites and feeds the receiver a false position. The receiver does not know it is being lied to, and reports a wrong but believable position.

Satellite signals are extremely weak (the satellites are about 20,000 km away), so a small nearby transmitter overpowers them. Since 2022 jamming and spoofing have become a daily event in parts of Europe, near conflict zones, ports and airports.

Most fleet software only displays the position it receives. It does not ask whether the position is true.

The objective

> Tell the operator, once per second, for every vehicle and drone, **how much its reported position can be believed**, and **why**, in plain words.

TrustNav does not replace the receiver. It sits next to it and acts as a second opinion.

What the tool does, in real life

Real situation	What happens without TrustNav	What TrustNav does
A van's tracker is spoofed so it appears 8 km away	The dispatcher sees a normal dot on the wrong street	Raises a **Position jump** alert within seconds: the vehicle cannot move 8 km in one second
A driver uses a €30 jammer to hide	The dot freezes or vanishes with no explanation	Flags **RF interference detected** from the receiver's jamming indicator and signal strength collapse, then tracks the **recovery**
A slow, careful spoofer pulls a vehicle 1 m/s off the road	Nothing looks wrong for minutes	The machine-learning layer notices the position and signal no longer match normal driving, in about 42 s in our tests (a fixed 150 m rule alone needs about 3.5 minutes)
A forged Galileo message	The receiver accepts it	**Galileo OSNMA** (the digital signature on Galileo data) fails verification, which is strong evidence of forgery
A drone on a survey mission is pushed off its plan	The pilot sees the drone "on course"	Compares the position with the mission route and the drone's physical limits

Who would use it

- **Fleet operators** (delivery, logistics, taxis, public transport) who need to know whether a vehicle is really where it says.
- **Insurers and cargo security**, where jamming is used in thefts.
- **Drone operators** (inspection, mapping, agriculture, medical delivery).
- **Critical services**: emergency vehicles, ports, rail, energy sites.

Design goals

1. **Explainable.** Every score comes with named reasons and the exact value and threshold that triggered it, so an analyst can check an alert by hand.
2. **Hybrid.** Three kinds of evidence work together, because each misses something the others catch: - *Rules*: physical limits that are impossible to break (speed, jump distance, acceleration). - *Machine learning*: spots strange combinations that no single rule was written for. - *Galileo OSNMA*: cryptographic authentication supplied by the satellites.
3. **Few false alarms.** An operator who is alerted constantly stops listening. Normal data in our tests produces a false alarm rate of 0.44%.
4. **Responds, not only reports.** A fix the engine does not believe is refused; the position handed to dispatch, geofencing or flight control is estimated from the odometer along the route, and the operator gets an action: continue, verify or isolate.
5. **Same code for demo and real life.** The demo runs through the exact detector a real receiver would use, and the receiver adapter already speaks NMEA and u-blox UBX.
6. **Safe.** Our own attacks are simulated in software; the real ones are recordings made by others. No radio signal is ever transmitted.

Measured results

Precision 99.3 %, recall 96.1 %, false alarm rate 0.44 %, 7 of 7 attack types caught; the response layer cuts the worst operator-visible error from 8.0 km to 29 m. The full tables, scenario by scenario, and the replays on real recordings are in section 6.

Limits (honest)

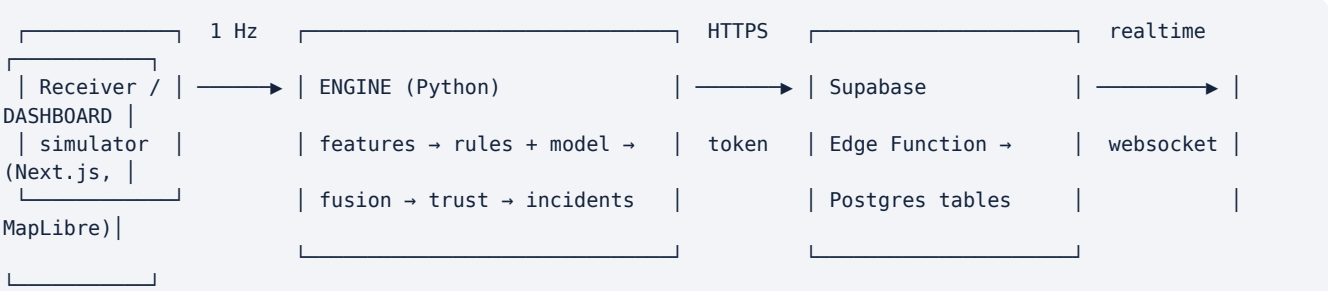
- The detection table is **simulated**. The real recordings replayed so far (one clean drive, two days of real jamming and spoofing) are evidence, not yet a statistic.
- The model learned from one simulator, so it must be re-calibrated on each real receiver; this was needed, and done, on both real recordings.
- The time rule needs a second clock (the phone's or the gateway's), or a spoofer that moves time backwards.
- It detects, explains and refuses bad fixes. It cannot remove a jammer or a spoofer from the air; only antenna hardware does that.
- Its estimate during a hold drifts with the odometer (about 3 % of the distance), so long holds need the operator to verify by other means.

Next steps

A real receiver and antenna, calibration on real normal drives, raw radio recordings of attacks through a software receiver, a second antenna, real OSNMA: the plan with effort and priorities is in section 8.

3 How it works, step by step

The big picture



Three parts, each in its own folder:

Part	Folder	Job
Engine	`engine/`	Scores each observation and decides what to alert on
Database and API	`supabase/`	Stores results, secures the write path, serves them live
Dashboard	`dashboard/`	Shows the fleet, the map, the reasons and the history

Step 1: An observation arrives (once per second)

A GNSS receiver reports, every second:

- position (latitude, longitude), speed, heading
- number of satellites tracked
- **C/N0**: signal strength per satellite, in dB-Hz
- **PDOP**: how well the satellites are spread across the sky (geometry)
- pseudorange residuals: how well the measurements agree with the computed position
- receiver clock bias and drift
- a jamming indicator (0 to 255)
- **Galileo OSNMA status**: verified, failed or lost
- optional inertial speed (from an IMU)

These come from three sources, all producing the same observation dictionary:

- the **simulator** (`simulator.py`): repeatable synthetic observations with or without attacks; every number in the results comes from it;
- a **real receiver** on a serial port or a log (`receiver.py`): NMEA and u-blox UBX messages folded into one observation per second;
- a **phone** (`phone.py` with the `/phone` page of the console): the phone's own fixes, position, speed, heading and accuracy, sent to the cloud once a second and pulled by the engine on a laptop. A phone gives no per-satellite C/N0, clock or OSNMA, so those inputs stay missing and the physics and route rules carry the score; the phone's real speed stands in for the odometer. The phone page can overlay a teleport, a slow drift or jamming on the real stream, in software only, so detection and the response layer can be shown on real data.

Step 2: Turn the observation into features (`features.py`)

One raw reading is hard to judge on its own. A position of 47.46, -0.55 says nothing. What matters is **how it compares with the last few seconds and with the expected route**.

`FeatureExtractor.update()` keeps a rolling window of the previous 5 observations and computes **36 features**, for example:

Feature	Meaning	Why it matters
`distance_m`	How far the fix moved since last second	A spoofer can make this huge
`derived_speed_mps` vs `reported_speed_mps`	Speed worked out from positions vs speed the receiver claims	Spoofed positions break this relationship
`acceleration_mps2` , `heading_rate_dps`	How violently it speeds up or turns	Vehicles have physical limits
`route_deviation_m`	Distance from the assigned route	Catches slow drift
`cn0_mean_dbhz` , `cn0_std_dbhz`	Signal strength and how uniform it is	A spoofer's signals are too strong and too uniform; jamming makes them collapse
`satellite_count_delta`	Sudden change in satellites tracked	Jamming or spoofing onset
`pdop` , `pseudorange_residual_m`	Geometry and measurement consistency	Degrade under attack
`clock_bias_delta_ns` , `clock_drift_delta_nsps`	Receiver clock behaviour	A spoofer disturbs the clock
`sensor_speed_mismatch_mps`	GNSS speed vs inertial speed	Two independent sources disagreeing
`osnma_failed` , `osnma_lost`	Galileo signature status	Cryptographic evidence
`missing_*` flags	A value is absent	Lets the system tell "missing" from "zero"

The extractor works one observation at a time, so it runs live exactly as it does on recorded data.

Step 3a: Rules (`rules.py` , thresholds in `config.py`)

Sixteen deterministic checks. Each one compares a feature with a named threshold:

`R_POSITION_JUMP` , `R_IMPOSSIBLE_SPEED` , `R_ACCELERATION_SPIKE` , `R_HEADING_DISCONTINUITY` , `R_ROUTE_DEVIATION` , `R_SATELLITE_SHIFT` , `R_CN0_SHIFT` , `R_DOP_DEGRADATION` , `R_PSEUDORANGE_RESIDUAL` , `R_CLOCK_ANOMALY` , `R_SENSOR_GNSS_MISMATCH` , `R_JAMMING_INDICATOR` , `R_OSNMA_FAILED` , `R_OSNMA_LOST` , `R_FIX_LOST` , `R_TIME_JUMP` .

- Each rule has a **weight** and a **severity**. Severity is 0.5 at the threshold and rises to 1.0 at three times the threshold, so a small exceedance counts less than a huge one.
- The rules' contributions are combined with a **noisy-OR**: independent evidence reinforces itself (`risk = 1 − ∏(1 − contribution)`).
- A **critical** rule (a jump over 4× the limit, an OSNMA failure, or a time jump) sets a **floor of 70** on the risk, so a clear forgery can never be averaged away.
- `R_TIME_JUMP` was added in version 2.2, after a recording of a real spoofer showed what the other fifteen could not see (Step 9). Receiver time is treated as a measurement: it is compared with the clock of the host the receiver is plugged into (a phone, a gateway), against a slowly adapting baseline, and a difference of more than 5 s is critical. Without a host clock the rule still fires when receiver time runs backwards against its own one-per-second count. Elapsed time everywhere else (speeds, accelerations, dead reckoning) is counted on that trusted clock, never on the receiver's time alone, and the position-jump limit scales with the time since the last fix.

- Thresholds sit where something becomes *physically impossible*, not where an attack becomes likely. That keeps false alarms low, at the cost of missing slow attacks. The model layer covers those.

Output: a list of reasons, each with the measured value and threshold, plus a rule risk from 0 to 100.

Step 3b: The machine-learning model (`model.py`)

Two detectors, trained **only on normal driving** (it never sees an attack, so it cannot just memorise attack shapes):

1. **Isolation Forest** (scikit-learn, 300 trees). Finds unusual *combinations* of inputs.
2. **Tail detector**. Finds a *single* extreme input, such as a position 90 m off the road while all signals look clean. It can say exactly which input was improbable.

Both scores are turned into a probability on a separate calibration set of normal runs, then combined (`p = min(2·min(p_forest, p_tail), 1)`) and converted to a risk: `risk = 100 · min(1, −log10(p) / 4)`. So **risk 50 means "about a 1-in-100 event during normal driving"**, which keeps the alarm line meaningful.

The model uses 19 of the 36 features (one per independent physical quantity). A trailing **median over 3 seconds** suppresses isolated spikes but lets sustained anomalies through.

When risk is 25 or more, the model **explains itself**: "Most unusual: route deviation 47.24 (normal 0.02 to 4.3)". That is the line you see in the dashboard.

Step 4: Fusion into a trust score (`pipeline.py → fuse`)

```
risk  = 1 − (1 − rule_weight·rule_risk) · (1 − ml_weight·ml_risk)       # noisy-OR
trust = 100 − risk
```

with `rule_weight = 0.6` and `ml_weight = 0.8`, chosen on **validation routes** (not the test routes).

Why not just average? Averaging a confident ML alarm (90) with a quiet rule score (20) gives 55, weaker than the ML evidence alone. Noisy-OR treats the two as independent witnesses: either can raise the alarm, and both together are stronger.

Worked example from the screenshot: rules 70, model 100 → `1 − (1 − 0.6·0.70)(1 − 0.8·1.00) = 1 − 0.58·0.20 = 0.884` → risk **88** → trust **12**.

Trust bands:

Trust	Status
75 to 100	Trusted
50 to 74	Degraded
25 to 49	Suspicious
0 to 24	Untrusted

Step 5: Incidents (`pipeline.py → IncidentTracker`)

An alert on a single second would flicker. So:

- An incident **opens** after trust stays below 50 for **3 consecutive seconds**.
- It **closes** after **5 clean seconds**.
- Severity comes from the lowest trust reached: below 35 is High, below 20 is Critical.
- The title leads with the strongest critical evidence, for example "Galileo authentication failed: 40 observations".

Step 5b: The response layer (`safeguard.py`)

Detection alone leaves the operator looking at a lie, so the score drives a response, the way an intrusion *prevention* system acts on what an intrusion *detection* system finds:

Trust	Mode	What downstream software receives	Action for the operator
50 or more, no critical rule	**accept**	the GNSS fix, unchanged	CONTINUE
below 50, or no fix	**hold**	an estimate: the last trusted fix advanced by the **odometer speed along the planned route** (a spoofer cannot touch the odometer); without a route, along the last trusted heading	VERIFY (confirm by other means before acting)
below 25, or a critical rule (OSNMA failed, 4× jump)	**hold**	the same estimate	ISOLATE (do not act on this device; a drone holds or returns on inertial navigation, a driver is contacted)

A hold ends only after **five clean seconds and a fix that rejoins the estimate** (within 60 m plus 3 % of the distance travelled while held), so a spoofer cannot end the hold by looking clean while still far away. After 60 clean seconds it releases with a tolerance ten times looser and reports the discrepancy, because an odometer estimate drifts too; a fix that is still kilometres away stays refused (found on a real recording: without this bound, a teleported fix that kept moving consistently was taken back after a minute).

Where the route doubles back on itself (the same street in both directions), `Route.project()` uses the direction of travel and continuity to pick the right leg, otherwise the estimate would advance the wrong way.

Measured on the held-out attacks (`protection` in `meta.json`): worst error on an ordinary screen 8,003 m, worst error of what the response layer handed on 29 m; jamming blank screen 100 s without, 0 s with; holds outside attacks 26 s in total over the whole catalogue.

Step 6: Send to the cloud (`stream.py` → `supabase/functions/ingest`)

`stream.py` runs the detector in real time (or faster with `--speed`) and posts the results every second to the **ingest Edge Function**:

1. `start_run` registers the devices and opens a run.
2. `frames` sends the scored frames and incident events (up to 100 per call).
3. `end_run` closes the run.

The function is the only write path. It checks an `x-trustnav-token` header against a **SHA-256 hash** stored in the database, so the real token is never stored, and it limits sizes and sanitises every field.

A real receiver instead of the simulator (`receiver.py`)

`python -m trustnav_engine.receiver --port /dev/ttyACM0` reads a GNSS receiver over USB (NMEA sentences GGA, RMC, VTG, GSA, GSV, and u-blox UBX messages NAV-PVT, NAV-CLOCK, NAV-STATUS, NAV-SAT, MON-HW), folds each second into the same observation dictionary the simulator produces, runs the same `Detector` , and posts to the same ingest endpoint. Inputs the receiver does not provide (OSNMA, inertial speed) stay `None` and the engine's `missing_*` flags take over. `--file` replays a saved log, `--jsonl` records everything for later calibration, and `--no-model` runs rules only until the model has been re-calibrated on that receiver's own normal drives. The receiver only listens; nothing is transmitted.

Step 7: Storage and security (`supabase/migrations`)

Tables: `devices` , `runs` , `frames` (one row per scored second, including the `response` JSON), `incidents` , `ingest_tokens` , `presenter_passcodes` , plus `demo_frames` and `demo_incident_events` for the built-in demo.

- **Row-level security**: everyone can *read* devices, runs, frames and incidents. Nobody can write except through the ingest function (service role).
- **Demo feed**: a `pg_cron` job (`demo_tick`) replays a precomputed fleet once per second, so the live demo works with no laptop attached. Starting and stopping it needs a **presenter passcode** (bcrypt-hashed, rate-limited after failed attempts).
- Old frames are pruned daily, and cron logs are trimmed every 10 minutes.

Step 8: The dashboard (`dashboard/`)

`useFleet.ts` is the data hook. If a run is `running` it subscribes to **Supabase realtime**; otherwise it loops a **recorded demo** (`public/data/demo_fleet.json`) so the page is never empty.

View	What it shows
Live	3D map of the fleet, trust per vehicle, alert feed, device detail with reasons and the response (accepted / held / isolated), timeline dock, report export
Replay	Ten recorded scenarios with a scrubber, the real path, the protected path the response layer handed on, and a narrated story mode
Results	The evaluation numbers, per-scenario outcomes, what the response layer prevented, and the two replays on real recordings (real attacks, a real drive with no attack)
Method	How it works and its limitations
Space-time	A 4D view: ground is the map, height is time, each track is a ribbon coloured by trust

The map is **MapLibre GL** with **OpenStreetMap** data (CARTO styles) and 3D buildings. Nothing about a position is hidden from the user: the *Why* panel lists every reason the engine produced.

Step 9: Evaluation (`evaluate.py`)

To avoid fooling ourselves, routes are split by role and never reused:

Split	Used for
train	Fit the Isolation Forest (normal driving only)
validation	Choose fusion weights
held-out	Every reported number

The simulator is deterministic (seeded), so every result can be reproduced exactly.

Two more commands leave the simulator and replay public recordings of real receivers through the same adapter and detector:

- `realcheck.py`: a clean 27.6-minute drive of a u-blox ZED-F9K with a survey-grade reference. It measures false alarms on a real receiver (none from the rules), shows that the simulator-trained model must be re-calibrated on the receiver in use, and overlays software attacks on real noise.
- `attackcheck.py`: recordings of **real jamming and spoofing** made in a car at Jammertest 2024 (VAARA dataset, University of Vaasa). A phone was captured by a spoofer for 26.5 minutes; the fifteen rules first published alerted after 21 min 49 s and refused 5 of 909 captured fixes. With `R_TIME_JUMP` the alert comes in the same second and all 909 are refused, with no alert in the clean periods. A u-blox driven past real jammers is alerted in 15 of 20 exposures by the re-calibrated hybrid and in 1 of 20 by the rules alone, because its NMEA stream carries no jamming indicator. The files are downloaded on demand (SHA-256 checked) and never stored in the repository.

4 The console, in plain words

The live console is the part a jury member sees first, so this section says what each element means in everyday words. The same text is available inside the console by pressing `?`.

What it shows. Every second, for each van and drone, TrustNav checks whether its satellite position can be believed and gives it a score from 0 (certainly false) to 100 (certainly true). Below 50 it refuses the position and tracks the vehicle by its odometer instead, so nothing downstream acts on a lie.

The map. A ring around each vehicle is its trust score drawn as a gauge: green is trusted, amber has minor doubts, orange is suspicious, pink is untrusted. A hollow marker labelled "refused" is the position the receiver claims and TrustNav is not passing on. A teal shield is where TrustNav believes the vehicle really is, worked out from its odometer along its route. The dashed line between them is the lie, with its length; in the teleport attack it is 8 km. The coloured trail is the last 90 seconds of reported positions.

The panels. The strip at the top left shows the fleet's average trust, how many vehicles are in each band, the open incidents, and how many positions are being refused with the size of the worst lie. The alerts panel at the top right has one card per incident with the last minute of trust, the action taken (verify or isolate) and a tick to acknowledge it. Clicking any vehicle opens its file: first a verdict in plain words, then the evidence (each line names what was measured and the limit it crossed), then what to do.

The three things that go wrong. *Jamming*: a noise transmitter drowns the satellites and the receiver loses its position; TrustNav keeps an estimate from the odometer, so the screen never goes blank. *Spoofing*: a transmitter imitates the satellites and feeds a false position, either a jump or a slow drift; TrustNav notices the impossible motion, the too-perfect signal or the drift off the road, and refuses the position. *Forgery*: fake Galileo navigation data; Galileo signs its messages (OSNMA), so the signature fails and the forgery is caught within seconds.

Presentation mode (key `P`) makes the camera visit every platform, incidents first, with a caption, for a jury room or a wall screen. **Replay** plays ten recorded attacks with a narrated story. **Results** has the numbers. **Method** explains how it decides and how it differs from existing products.

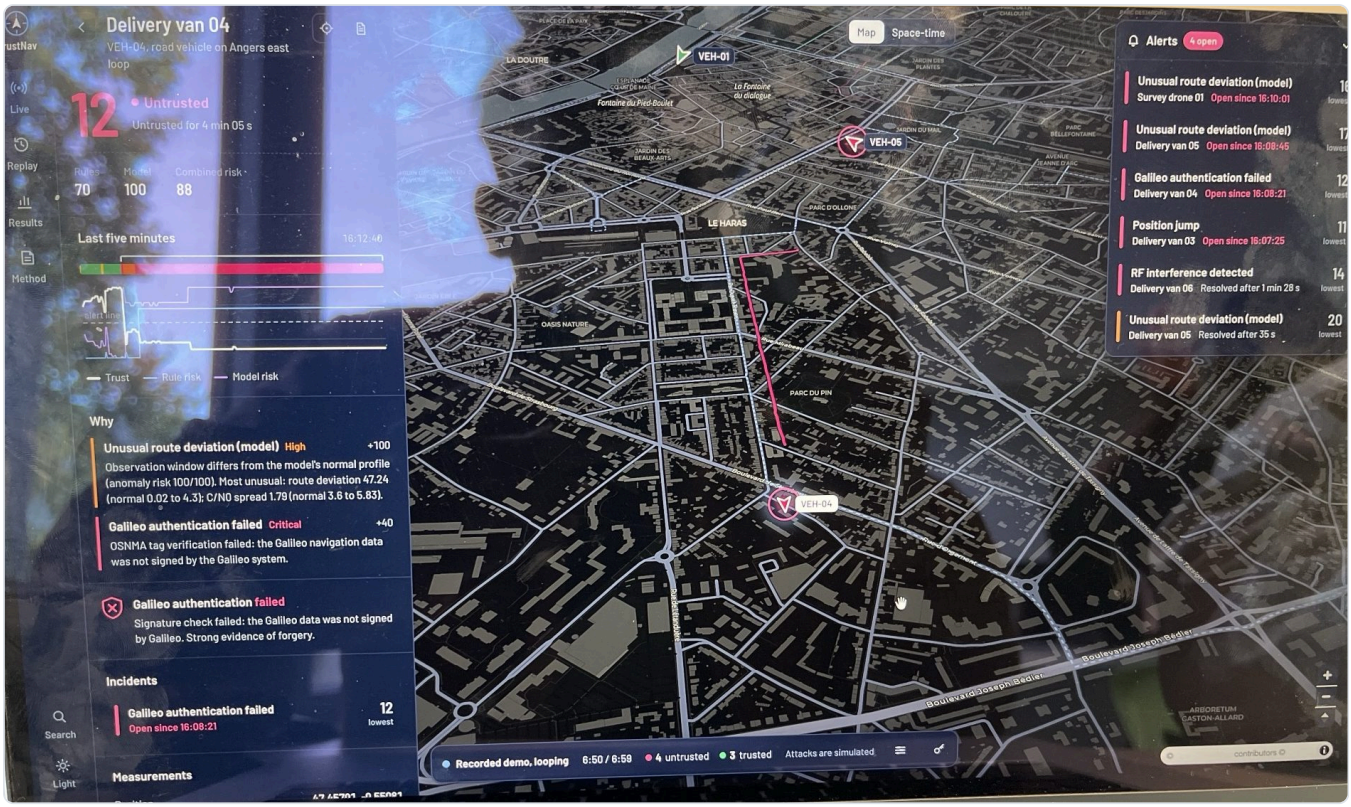


Figure 1. The deployed console (trustnav.vercel.app) photographed on 29 October 2026: Delivery van 04 under a Galileo forgery attack, untrusted for 4 minutes, with the forged track in pink over the real streets of Angers. Later versions add the trust rings, shields and plain-words verdicts shown below.

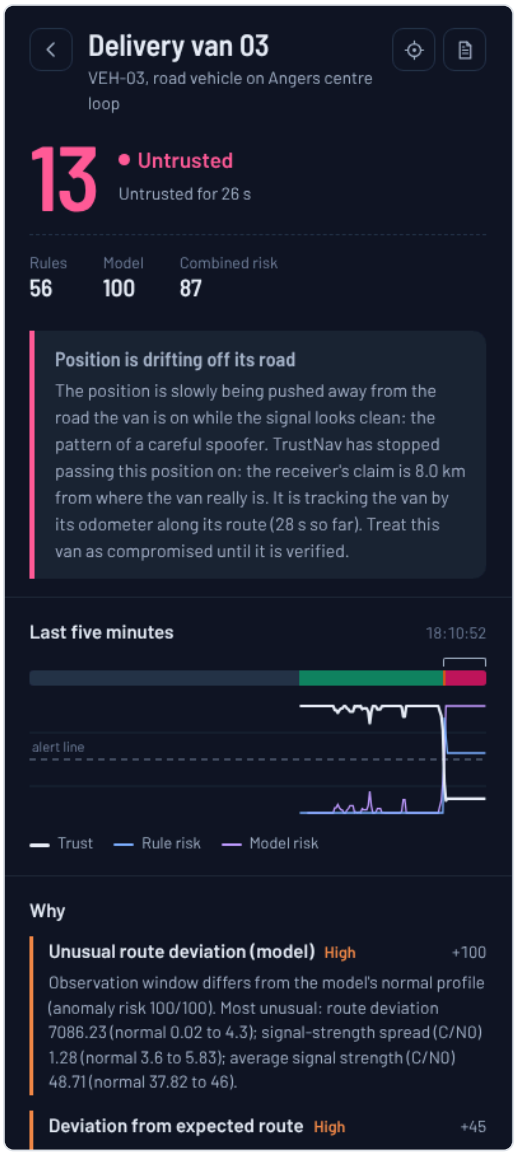


Figure 2. A vehicle's file: the score, the verdict in plain words, the last five minutes, and the evidence.



Figure 3. The response layer on the map: hollow refused fixes, teal shields where the vehicles really are, and the dashed lies ("8.0 km refused"). The basemap is simplified in this test render.

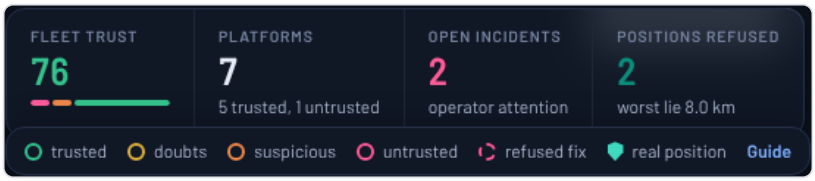


Figure 4. Fleet at a glance.

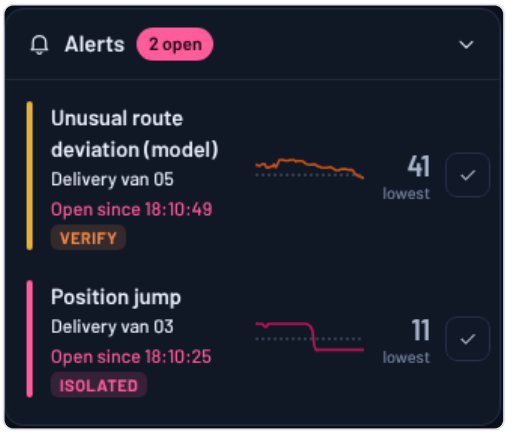


Figure 5. Alert cards with sparklines, the action taken and acknowledgement.

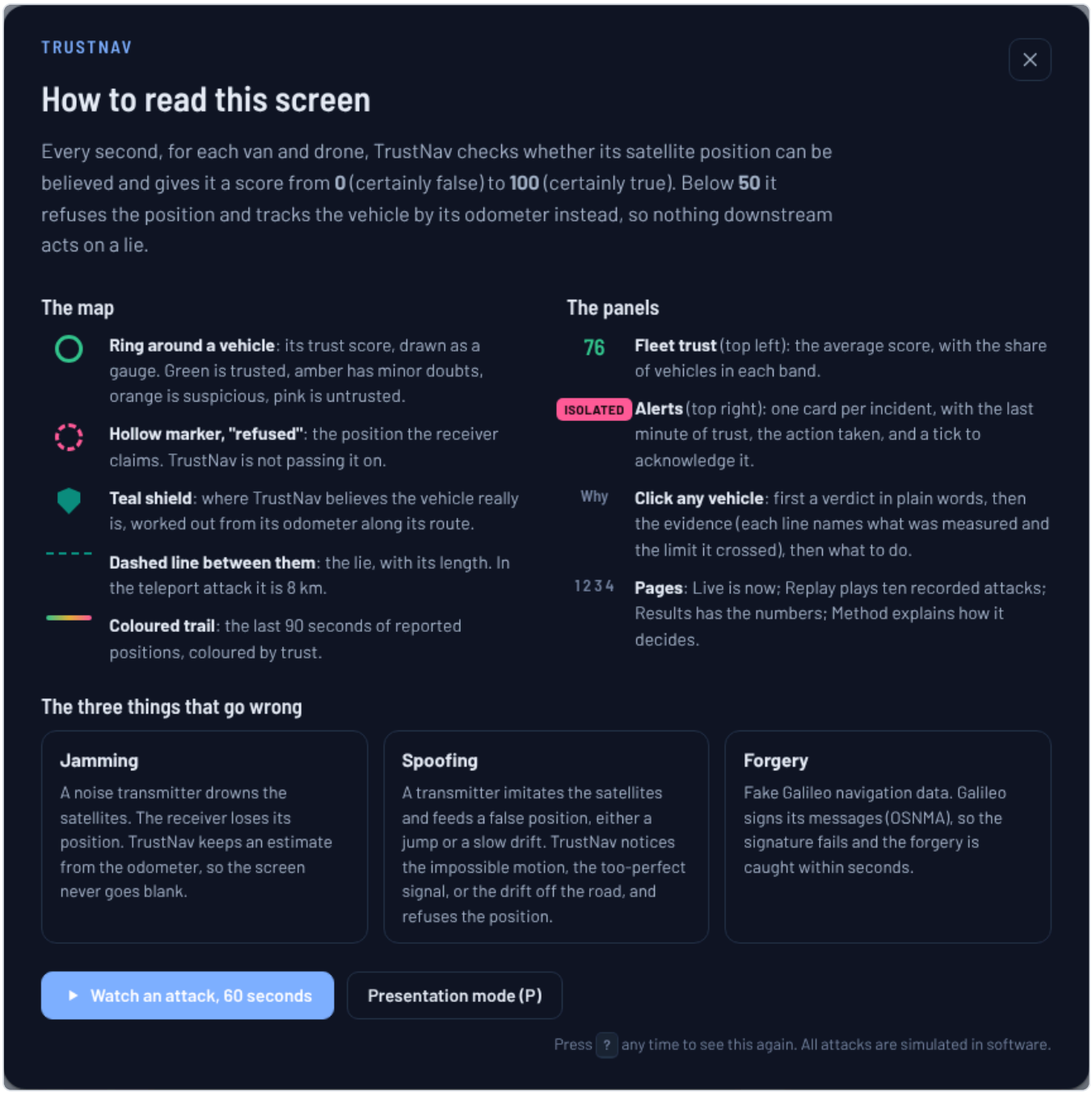


Figure 6. "How to read this screen", one keypress away inside the console.

5 Code guide: every file and what it does

Folder map

```
trustnav/
├── README.md                      overview, results, how to run
├── docs/                          this documentation
├── engine/                        Python detection engine
│   ├── requirements.txt           numpy, scikit-learn, joblib, pytest
│   ├── models/current/            trained model (model.joblib, model.json) + chosen fusion weights
(fusion.json)
│   ├── tests/                     42 automated tests (engine, response layer, receiver adapter, phone,
time, real recordings)
│   └── trustnav_engine/
├── dashboard/                     Next.js web app
│   ├── app/                       page shell, global styles, icon
│   ├── components/                the screens and widgets
│   ├── lib/                       data hook, types, helpers
│   ├── public/data/               recorded data the page can play without a database
│   └── tests/e2e.py               109 end-to-end checks
└── supabase/
    ├── migrations/                database schema and functions, in order
    ├── functions/ingest/          the secure write endpoint
    └── load_demo_feed.sql         loads the demo data
```

Engine (`engine/trustnav_engine/`)

File	Lines	What it does
`config.py`	121	**All** thresholds and weights in one place so a reviewer can question each one. Platform profiles (`ROAD_VEHICLE` : 60 m/s, 8 m/s², 90 °/s, 250 m jump; `SMALL_UAV`), `RuleSettings` , `FusionSettings` , `TrustBands` , `IncidentSettings`
`geo.py`	53	Geodesy helpers: `haversine_m` (distance), `bearing_deg` , `offset` , `angle_diff_deg` , `LocalFrame` (lat/lon to metres)
`routes.py`	200	`Route` class: a polyline the vehicle should follow, with `distance_to()` and `project()` (arc-length position, direction- and continuity-aware where the road doubles back). Road routes were computed with OSRM on OpenStreetMap data and stored in `data/routes.json` . Each route has one **role**: train, validation or held-out
`simulator.py`	269	Deterministic synthetic observations (seeded). Normal driving with stops, corners and urban-canyon dips, plus attack overlays: `teleport` , `drift` , `signal` , `stealth_drift` , `osnma_forgery` , `jamming` , `drone_drift` . Every observation carries a ground-truth `attack` label. **Nothing is transmitted**
`scenarios.py`	95	Catalogue: `training_specs` (normal only), `validation_specs` , `heldout_specs` (the 10 demo scenarios), `FLEET_DEVICES` and `fleet_specs` (the 7-platform live demo)
`features.py`	295	`FeatureExtractor.update(obs)` : 36 features over a 5-observation rolling window, plus context. `_time()` keeps the trusted clock: elapsed time is counted on the host clock when there is one, otherwise on receiver time while it runs forward, and the difference is reported as `time_jump_s` . `MODEL_FEATURES` is the 19 used by the model
`rules.py`	149	`evaluate()` : the 16 rules, the last one `R_TIME_JUMP` (receiver time against the host clock or against its own count, critical). Returns reasons, rule risk, critical flag. `_severity()` scales 0.5 to 1.0
`model.py`	193	`TailModel` and `AnomalyModel` : Isolation Forest plus tail detector, calibration, `risk_from_p` , `explain()` , `save` / `load`
`pipeline.py`	268	`trust_status` , `fuse()` (noisy-OR), `IncidentTracker` , and `Detector.process(obs)` which turns one observation into one scored frame, then asks the response layer what to hand on

File	Lines	What it does
`safeguard.py`	174	**Response layer.** `Safeguard.update(frame, obs, clock)`: accept / hold / release logic, stepped on the trusted clock so skipped seconds do not shorten the distance, route-constrained dead reckoning from the odometer, rejoin tolerance, CONTINUE / VERIFY / ISOLATE actions
`receiver.py`	428	**Real receiver adapter.** NMEA (GGA, RMC, VTG, GSA, GSV) and UBX (NAV-PVT, NAV-CLOCK, NAV-STATUS, NAV-SAT, MON-HW) parsers, tolerant of a sentence glued to a binary frame; `EpochBuilder` folding one second into one observation (a missing satellite count stays unknown, a second without time is marked), serial port or log file input, `--no-model`, `--jsonl`
`evaluate.py`	259	Train, pick fusion weights on validation routes, score held-out routes, compute precision, recall, F1, false-alarm rate, detection delay; `protection_report()` measures reported vs protected position error against ground truth
`export.py`	99	Writes everything the dashboard needs to JSON
`demo_feed.py`	59	Precomputes the 7-platform fleet with the real detector for the cloud demo
`realcheck.py`	301	**Real drive, no attack.** Replays a recorded drive (default: the public u-blox ZED-F9K recording in `data/real/`, with its NovAtel SPAN reference) through the adapter and detector: rules alone, simulator model and C/N0+sats baseline on the whole drive; the model re-calibrated on the first part of the drive and tested on the rest; teleport, drift and jamming overlays (`RealOverlay`) on the test segment with detection delay and worst raw / protected error against the reference; the reference track is the route, its speed over ground the odometer. `--out` writes the results, with the drive as a polyline (`track_summary`), to `paper/figures/realcheck.json` and `dashboard/public/data/realcheck.json`
`attackcheck.py`	538	**Real attacks.** Replays the VAARA recordings of Jammertest 2024 (a u-blox receiver and a phone in a car, under real jamming and spoofing). `fetch()` downloads the six files on demand with a SHA-256 check into a git-ignored folder; `ublox_observations()` and `phone_observations()` turn the logs into observations (the phone's lines carry its own system clock as `host_t`); `reference()` is the receiver the spoofer did not capture; `phone_session()` compares the rules as first published, with an odometer, with the re-calibrated model and with the time rule, and measures what the response layer hands on; `ublox_session()` counts jamming exposures alerted and false alerts by signal state. `--out` writes `paper/figures/attackcheck.json` and `dashboard/public/data/attackcheck.json`
`phone.py`	193	**Phone bridge.** Pulls the fixes the `/phone` page stored in the cloud (`phone_pull`), maps them to observations (signal inputs stay None, the phone's real speed stands in for the odometer), applies the attack overlay chosen on the phone (`Overlay`: teleport 8 km east, drift 0.7 m/s, jam = no fix), scores them with `Detector` and posts frames as a `receiver` run. `--route`, `--model`, `--from-start`, `--dry-run`
`stream.py`	103	Live runner: one observation per second per device, posts to the ingest endpoint. `--dry-run`, `--speed`

The core function: `Detector.process`

```
f, ctx = self.fx.update(o)                                # 1. features
reasons, rule_risk, critical = evaluate(f, ctx, ...)    # 2. rules
ml = median(self.ml_hist)                                 # 3. model risk (3 s trailing median)
hybrid = fuse(rule_risk, ml, st.fusion, critical, floor)  # 4. noisy-OR fusion
trust = 100.0 - hybrid                                    # 5. trust score
frame["resp"] = self.safeguard.update(frame, o, clock)  # 6. response: accept or hold, protected position,
action
events = self.incidents.update(frame)                    # 7. open / update / resolve incidents
return frame, events
```

The frame returned has: `trust`, status, rule risk (`rr`), model risk (`mr`), hybrid risk (`hr`), position, OSNMA state, the top 5 reasons (`why`), 16 display features, and `resp` = `{mode, lat, lon, held_s, action, gap_m, source}`.

The response layer: `Safeguard.update`

```
acceptable = has_fix and trust >= 50 and not critical
if holding:
    advance the estimate by odometer_speed × dt along the route (or heading)
    if acceptable and clean streak >= 5 and |fix − estimate| <= 60 m + 3 % of distance held:  release
elif acceptable:  accept (remember this fix and its arc-length position on the route)
else:             start a hold from the last accepted fix
action = CONTINUE if acceptable else ISOLATE if (critical or trust < 25) else VERIFY
```

Tests (`engine/tests/`)

`test_engine.py` (9): simulation is deterministic; attack labels start on time; noisy-OR is stronger than either witness (40 and 40 give 64); missing ML falls back to rules; a critical rule pins the risk floor; a rule is silent just below its threshold; severity scales with the exceedance; OSNMA unknown is never a failure; trust bands map correctly.

`test_safeguard.py` (9): a trusted fix passes through; a spoofed fix is held and the estimate follows the odometer; a hold does not release while the fix is still far away, not even after a long clean streak; a long clean streak does release a fix that is merely drifted (within ten times the rejoin tolerance), reporting the discrepancy; a clean fix that rejoins releases the hold; no fix still gives a position; action bands; route projection picks the right leg where the road doubles back; the route odometer tracks a jammed vehicle within 2 m over a minute.

`test_phone.py` (5): a phone row becomes an observation with the signal inputs missing and the real speed as odometer; a no-fix row has no position; the teleport, drift and jam overlays move or remove the fix as specified; a steady walk is trusted and a teleport is caught in the same second (position-jump rule, ISOLATE, hold) with the protected estimate staying within 30 m of the real walk; a jam overlay still yields a dead-reckoned estimate.

`test_realcheck.py` (3): the real recording parses into one observation per second with primary-band C/N0; the rules raise nothing on its opening minutes and the track summary labels the fit, calibration and test segments in order; the jam overlay degrades C/N0 then removes the fix.

`test_receiver.py` (8): NMEA sentences parse; two epochs become two observations; a no-fix epoch has no position; UBX frames and the five messages parse (jamming indicator, clock, satellites, PVT); the live detector scores receiver observations; rules-only mode has no model risk; a sentence glued to a binary frame is still read; the satellite count stays unknown when the sentence that carries it is dropped.

`test_time.py` (5): receiver time set back is critical for as long as it lasts against the host clock; time running backwards is caught without a host clock, and a gap is not; a step of the host clock is not blamed on the receiver; the position-jump limit scales with the time since the last fix; a hold is stepped on the clock, so skipped seconds do not shorten the distance.

`test_attackcheck.py` (3): the phone's log lines carry the phone's own clock; a two-hertz log is thinned to one observation a second; seconds are sorted into test, no-test, margin and clean. The recordings themselves are not needed to run the tests.

Supabase (`supabase/`)

File	What it does
`migrations/..._trustnav_core_schema.sql`	Tables `devices`, `runs`, `frames`, `incidents`, `ingest_tokens`, `presenter_passcodes`, `demo_frames`, `demo_incident_events`, indexes, row-level security (public read only)
`migrations/..._demo_feed_functions.sql`	`start_demo_feed`, `stop_demo_feed`, `check_presenter`, `demo_tick` (runs every second via `pg_cron`), `prune_old_frames` (daily)
`migrations/..._demo_feed_fetch.sql`	`request_demo_feed` using `pg_net` to fetch the demo data
`migrations/..._presenter_rate_limit.sql`	`presenter_failures` table and rate limit on the passcode
`migrations/..._cron_log_cleanup.sql`	`trim_cron_logs` every 10 minutes

File	What it does
`migrations/..._phone_observations.sql`	`phone_observations` table for the fixes the `/phone` page sends (row-level security on, no public policy: only the edge function reads and writes it); the daily prune also drops phone fixes older than 3 days
`migrations/..._response_layer.sql`	Adds the `response` column to `frames` and copies it in `demo_tick`
`functions/ingest/index.ts`	The only write path: POST only, token check by SHA-256 hash, `start_run` / `frames` / `end_run`, size limits (100 frames per call), input sanitising, upserts that ignore duplicates, `resp` stored as `response`. Also `phone_obs` (the phone page stores fixes) and `phone_pull` (the laptop bridge reads them after a cursor), with CORS so the page can call it from the browser
`load_demo_feed.sql`	Loads the precomputed demo into `demo_frames`

Security model: the browser can only read. All writes pass through the ingest function with a secret token that exists only in `engine/.env.local` (never committed; see `.gitignore`).

Dashboard (`dashboard/`)

Stack: Next.js 15, React 19, TypeScript, MapLibre GL, supabase-js, self-hosted Barlow fonts. No UI framework, hand-written CSS.

`lib/`

File	What it does
`useFleet.ts`	The data hook. Source is `connecting`, `live` (Supabase realtime) or `loop` (recorded demo). Returns devices, frames, incidents and the clock
`types.ts`	Shared types (`Frame`, `Incident`, `ReplayData`)
`supabase.ts`	Read-only client using the public key
`story.ts`	The narrated story beats and camera moves
`i18n-explainer.ts`	The narrated explainer's nine scenes in ten languages, with the UI strings and placeholders
`i18n-hero.ts`	The opening screen in ten languages (small, ships in the first load)
`hero-path.ts`	Geometry of the opening scene (the recorded track around the teleport, the planned route, the offset of the lie), generated from the teleport scenario in `public/data/replay.json`
`useVoice.ts`	`useVoicePref` (voice on/off, remembered) and `useNarrate` (reads a caption aloud when it changes) for the story and presentation captions
`speech.ts`	Browser speech synthesis: voice choice per language (natural voices first, Hindi as the fallback for Marathi), speak with a watchdog, stop
`plain.ts`	The plain-language layer: `plainVerdict` (one or two sentences per platform from the evidence codes and the response), `plainFleet` (the fleet headline) and `gloss` (engineering terms to everyday wording)
`format.ts`	Number, time and status formatting
`useTheme.ts`, `useShortcuts.ts`, `useWide.ts`	Dark mode, keyboard shortcuts, responsive layout

`components/`

File	What it does
`Console.tsx`	The shell: rail navigation, hash routing, command palette commands, shortcuts, hero and tour
`LiveView.tsx`	Live page: fleet list, device detail (with the response line and the operator playbook), fleet KPI strip, alert feed with sparklines, actions and acknowledgements, status strip, toasts, timeline dock, presentation mode
`MapView.tsx`	MapLibre map: light/dark styles, 3D buildings, markers with a trust ring and score (hollow when the fix is refused), smooth gliding, fading trails, follow camera, the protected path, shield markers with the size of the refused lie and the dashed link to the reported fix, framing of a whole lie on selection
`ReplayView.tsx`	Scenario replay with scrubber, ghost "real position", story mode
`SpaceTime.tsx`	Canvas 4D view: ground grid, time axis, trust-coloured ribbons
`ResultsView.tsx`, `MethodView.tsx`	Evaluation numbers (headline scoreboard, comparison tables, weight grid) and method/limitations; both load lazily
`AttackCheck.tsx`	"On recordings of real attacks" on the Results page: the error of the position a captured phone reports against the error of the position handed on (log scale), the four detectors compared (first alert, captured fixes refused), and three tiles for the u-blox under real jamming and beside the spoofer, read from `public/data/attackcheck.json`
`RealCheck.tsx`	"On a real drive with no attack" on the Results page: the track of the public u-blox drive (fit, calibration, test), false alarms before and after re-calibration, and the three overlaid attacks, read from `public/data/realcheck.json`
`FusionLab.tsx`	"Try it" on the Method page: two sliders (rule risk, model risk), the trust an average would give against the noisy-OR fusion with the chosen weights, and whether the fix is accepted or refused
`HeroScene.tsx`	The opening scene on a canvas: the van on its recorded route, the teleport that throws the reported position 8 km away, the trust gauge; geometry in `lib/hero-path.ts` (from the teleport scenario); a still frame under reduced motion
`Instruments.tsx`	Trust tape, trust chart, gauges, the OSNMA line and the response line (accepted / held / isolated, with what to do)
`Report.tsx`	Per-device incident report (print to PDF)
`Guide.tsx`	"How to read this screen": legend, panels and the three attacks in plain words (`?`)
`Explainer.tsx`	The narrated explainer: scene playback, spotlight on the target panel, caption, language menu, keyboard; drives the live view through `ExplainApi` (fit, select a trusted or a refused platform, open alerts)
`CommandPalette.tsx`, `Toasts.tsx`, `Hero.tsx`, `Tour.tsx`, `Icons.tsx`	Search, notifications, opening screen, guided tour, icons

Tests (`dashboard/tests/e2e.py`)

109 Playwright checks: every view, the opening scene and its link-preview image, the results scoreboard, the real-attacks section and the real-drive section, the fusion lab, interactions, keyboard shortcuts, command palette, report, story mode, space-time, the response layer in the replay panel, map and results, the comparison table, the live KPI strip, trust-ring markers, live shields and lie labels, alert sparklines, actions and acknowledgements, the playbook, presentation mode, phone layout, accessibility (axe), and a JavaScript size budget.

Deployment

- **GitHub**: this repository (`main`).

- **Vercel**: builds the `dashboard/` folder on every push to `main` and serves https://trustnav.vercel.app.
- **Supabase**: hosts the database, realtime and the ingest function.
- **Secrets**: the ingest token lives only in `engine/.env.local` and in the database as a hash.

How to run a real receiver (Phase 2)

The adapter is written and tested; only the hardware is missing.

```
cd engine
python -m pip install pyserial
python -m trustnav_engine.receiver --port /dev/ttyACM0 --device VAN-REAL --name "Real van" --no-model --
dry-run   # first look
set -a; source .env.local; set +a
python -m trustnav_engine.receiver --port /dev/ttyACM0 --device VAN-REAL --name "Real van" --no-model --
jsonl drive1.jsonl
```

The dashboard shows it as a real device (run mode `receiver`) next to the simulated ones. `--no-model` is deliberate: the model's "normal" is the simulator's; after a few recorded drives the thresholds and the model are re-calibrated on the real receiver (see ROADMAP.md (section 8)).

6 Results

All numbers come from the ten held-out scenarios (3,800 one-second observations) on routes the model never saw in training. The data is synthetic (one simulator produces every split), so these are an upper bound on real-world performance; the replays on real recordings are at the end of this section.

Detection

	Rules only	Model only	Combined (TrustNav)
Precision	100.0 %	97.9 %	**99.3 %**
Recall	27.9 %	90.1 %	**96.1 %**
F1	0.437	0.939	**0.977**
False alarm rate	0.00 %	1.33 %	**0.44 %**
Mean time to detect	9.0 s	6.57 s	**6.71 s**

Rules never cry wolf but miss slow attacks; the model catches subtle changes but is noisier; combined, each covers the other's blind spot. The fusion weights (rules 0.6, model 0.8) were chosen on separate validation routes, never on the results above.

Scenario by scenario (time from attack start to the first alert)

Scenario	Rules	Model	Combined	False alerts
Normal drive: Angers centre loop	normal run	normal run	normal run	0
Normal drive: Angers east loop	normal run	normal run	normal run	2
Teleport spoof: 8 km position jump	0 s	1 s	0 s	0
Gradual drift spoof: slow lateral takeover	missed	1 s	1 s	0
Signal anomaly: spoofer onset	missed	1 s	1 s	0
Stealth drift: clean signal, OSNMA verified, wrong position	missed	40 s	42 s	0
Galileo OSNMA forgery: authentication fails	15 s	1 s	1 s	0
Jamming: fix lost, then recovered	12 s	1 s	1 s	6
Drone survey: Lac de Maine, normal	normal run	normal run	normal run	2
Drone survey: spoofed off mission	missed	1 s	1 s	0

What the response layer prevents

Measured against simulator ground truth during each attack: the worst position an ordinary fleet screen would have shown, against the worst position the response layer handed downstream.

Scenario	Ordinary screen, worst error	With response layer, worst error	Hold began after	Held	Holds outside the attack
Normal drive: Angers centre loop	normal run	–	–	0 s	0 s
Normal drive: Angers east loop	normal run	–	–	6 s	6 s
Teleport spoof: 8 km position jump	8,003 m	4 m	same second	230 s	0 s

Scenario	Ordinary screen, worst error	With response layer, worst error	Hold began after	Held	Holds outside the attack
Gradual drift spoof: slow lateral takeover	211 m	16 m	1 s	249 s	0 s
Signal anomaly: spoofer onset	7 m	21 m	1 s	229 s	0 s
Stealth drift: clean signal, OSNMA verified, wrong position	179 m	20 m	42 s	228 s	0 s
Galileo OSNMA forgery: authentication fails	130 m	15 m	1 s	229 s	0 s
Jamming: fix lost, then recovered	blank 100 s	2 m	1 s	123 s	14 s
Drone survey: Lac de Maine, normal	normal run	–	–	6 s	6 s
Drone survey: spoofed off mission	257 m	29 m	1 s	229 s	0 s

Worst spoofing error on an ordinary screen: **8,003 m**; worst error of what the response layer passed on: **29 m**. Jamming blank screen: **100 s** without, **0 s** with. Holds outside attacks (a brief dip on a normal run, or the seconds after an attack ends before the fix rejoins the estimate): 26 s in total, each at an error of a few metres. A hold costs a few metres of odometer error even when the attack did not move the position (signal anomaly).

On recordings of real attacks

The tables above come from a simulator. What follows does not. At Jammertest 2024 on Andøya, Norway, the authorities let jammers and spoofers transmit, and the University of Vaasa recorded a u-blox receiver and a phone in a car through it (the VAARA dataset). The recordings were replayed unchanged through the same adapter and detector (`python -m trustnav_engine.attackcheck`).

Real spoofing. A phone in a car was captured by a real spoofer for 26.5 minutes: its clock was set back 60 minutes and its position 34 m aside, and when the car drove off the phone stayed where the spoofer had put it, ending 10.2 km from the truth. The u-blox beside it was not captured and serves as the reference.

Detector	First alert	Captured fixes refused
The 15 rules as first published	after 21 min 49 s	5 of 909
The same, with the car's speed as odometer	after 21 min 49 s	32 of 909
The same, with the model re-calibrated on this phone	same second	41 of 909
16 rules: receiver time checked against the phone's own clock	same second	909 of 909

The first version missed this attack: nothing in the spoofed stream was physically impossible. What the spoofer could not hide was time. With the sixteenth rule there is no alert in the 37 clean minutes before the capture nor in the 9 minutes declared free of tests, and with the car's speed as odometer and its track as route the position handed on stays within 45 m (median 2.3 m; a best case).

Real jamming. The u-blox drove past parked jammers and started inside a jammed area: 20 exposures. The rules alone alerted on 1, because its NMEA stream carries no jamming indicator; the hybrid, its model re-calibrated on the 38 minutes recorded before the first test, alerted on 15, about 10 s in, and marked all 20 as degraded, with 0.8 % false alerts on clean-signal seconds. Beside the spoofer the same u-blox was not captured: it was shown as degraded for 99 % of 62 minutes, and its position was kept.

Limits: two days of one campaign on two devices; the time rule needs a host clock, or a spoofer that moves time backwards. The recordings also exposed four defects in the adapter and the response layer, all fixed and tested.

On a real drive with no attack

A public 27.6-minute recording of a u-blox ZED-F9K driving a loop in Mountain View, with a survey-grade reference track (`python -m trustnav_engine.realcheck`). The rules alone raised **0** false alarms in the whole drive (lowest trust 90); the usual signal-strength check raised 86 s. The model trained on the simulator alerted on 89 % of the seconds, so it must be re-calibrated on the receiver in use: re-calibrated on the first 19 minutes it raised 0 alerts in the remaining 8.5 minutes, and software attacks overlaid on that segment were caught after 0, 35 and 1 s (teleport, drift, jamming).

Read these numbers with care

- All observations are deterministic synthetic output. One simulator produces every split, so these numbers are an upper bound on real-world performance.
- Attack overlays are textbook versions: a careful adversary who matches C/N0 spread and clock behaviour would be harder to detect.
- Route deviation assumes the vehicle follows an assigned route (delivery fleet, survey drone). A driver who changes route will look like drift.
- One observation is one evaluation unit. A long attack contributes many correlated rows, so per-observation scores are not per-attack accuracy.
- OSNMA status is simulated. A relay (meaconing) attack can pass OSNMA, which the stealth drift scenario shows.
- On a public 27.6-minute u-blox ZED-F9K drive recording (python -m trustnav_engine.realcheck) the rules raise no false alarm, but the model trained on the simulator alerts on 89 % of the seconds: it must be re-calibrated on the receiver actually used. Re-calibrated on the first 19 minutes of that drive it raises no alert in the remaining 8.5 minutes and catches overlaid teleport, drift and jamming in 0, 35 and 1 s.
- On recordings of real attacks (Jammertest 2024, python -m trustnav_engine.attackcheck) the fifteen rules first published missed a phone captured by a spoofer for 22 minutes: its stream was physically consistent and only its clock had been set back. The sixteenth rule, receiver time against the host clock, refuses every captured fix from the first second. Real jamming is seen by the re-calibrated model and not by the rules when the receiver sends no jamming indicator, and a lost fix alone is held, not called an attack.

7 How TrustNav differs from what exists today

The tutor's question: *why is this different from what is already on the market?*

Short answer: existing tools either live **inside one receiver** and output a flag with no explanation, or they are **fixed monitoring stations** for sites and countries. None of the ones we found gives a fleet operator a per-vehicle, per-second, explained trust score with a response. That gap is where TrustNav sits. The sections below say what each existing approach does, with sources, and then what TrustNav adds and what it does not.

Facts below were checked on 3 November 2026 against the sources listed. Where a vendor does not publish something (prices, internals), we say so rather than guess.

1. Detection inside the receiver chip

u-blox (the chips in most trackers, drones and cheap USB receivers) report a *spoofing state* (0 unknown, 1 none, 2 indicated, 3 multiple indications) and a *jamming state* to the host. Newer firmware (ZED-F9P HPG 1.50, July 2024) adds a jamming log, Galileo OSNMA and per-band flags. - Runs inside the chip. Covers jamming and spoofing. One receiver only. No explanation: the output is a state code, and u-blox notes that state 1 "does not prove the receiver is not spoofed". The method depends on first locking to genuine signals; a receiver that starts inside a spoofed field may never notice. At Jammertest 2025 a u-blox receiver raised both jamming *and* spoofing flags during a jamming-only run. - Typical boards: NEO-M9N about 60 to 70 EUR, ZED-F9P about 190 to 280 EUR (third-party shops).

Septentrio AIM+ is a similar built-in layer on higher-end survey and timing receivers: notch filters against jammers, a spectrum view, spoofing and meaconing flags from timing and cross-constellation inconsistencies, OSNMA in the top tier. One receiver, firmware-level, no fleet view, prices not published.

What this means for TrustNav. These flags are *inputs*, not competitors. The receiver adapter (`engine/trustnav_engine/receiver.py`) reads the u-blox jamming indicator and spoofing state and passes them to the engine, which combines them with motion, route and signal evidence and explains the result. A receiver flag on its own tells the operator nothing about *which* vehicle, *since when*, *how far off* or *what to do*.

2. Fixed monitoring stations

GPSPatron sells probes (1 to 3 antennas, rack-mounted) that stream raw observables to a cloud that classifies jamming and spoofing types, with charts and a claimed 0.6 s detection latency. It is designed for sites and nationwide networks (1,000+ probes claimed); the three-antenna probe needs at least 1 m of antenna separation. Prices on request.

Safran (Orolia) BroadShield is a software library sold inside time servers, or embedded into a customer's product with vendor support, detecting jamming types and spoofing indicators (position and time jumps, data anomalies). No fleet layer is described.

What this means for TrustNav. Fixed probes protect a *place* (a port, an airport, a data centre). TrustNav protects a *moving platform*, with no extra antenna, using the receiver the vehicle already has.

3. Hardware that mitigates jamming

infiniDome GPSdome / OtoSphere are boxes placed between the antenna and the receiver that null-steer jammers using two or more antennas (about 20 dB attenuation for GPSdome, dual band in GPSdome 2). They are primarily jamming *mitigation*; the spoofing claim appears only in an older fleet demonstration. Prices not published.

What this means for TrustNav. Nothing in software can remove a jammer from the air. TrustNav is explicit about this: during jamming it detects, keeps an estimated position from the odometer, and tells the operator. A null-steering antenna is a complementary purchase, not an alternative.

4. Software-only spoofing detection products

Regulus Cyber Pyramid GNSS (2019 to 2020) was a software-only spoofing detector with machine learning, integrated into Harman's automotive security product. The company still exists but now markets counter-drone products; Pyramid GNSS no longer appears on its site, and we found no public details of its method or explanations.

Fleet telematics platforms. Geotab's newer GO devices log a `GpsJammingDetected` event when the GPS module's own jamming detector fires; the operator can build a report on it. We found no spoofing detection claim from Geotab, and nothing at all about jamming or spoofing on Samsara's or Webfleet's sites. Navixy exposes a "GPS jamming" rule that depends on the tracker's own flag.

What this means for TrustNav. The fleet software market mostly *relays the receiver's own flag*, when it relays anything. There is no per-second trust score, no explanation and no response layer in what we could find.

5. Galileo OSNMA (authentication)

Galileo OSNMA has been operational since 24 July 2025 (EU Commission / EUSPA). It signs the Galileo navigation data so a receiver can tell authentic Galileo messages from forged ones; it is mandatory in smart tachographs on new EU trucks and buses from December 2025.

What it does not do, in the EU GNSS Service Centre's own words: it gives "no protection against jamming" and "does not prevent spoofing", it only lets a receiver flag unauthenticated data. GPS data is not authenticated by it. Researchers have shown replay (meaconing) attacks that pass OSNMA on real receivers (Wang et al., 2025).

What this means for TrustNav. OSNMA is one of the three evidence layers, not the whole answer. The stealth-drift scenario exists precisely to show a relay attack where OSNMA still verifies and only the motion evidence catches the attack.

6. Aviation interference maps

GPSJam and Flightradar24 colour the world by how many aircraft report degraded navigation accuracy (ADS-B NACp or NIC). They show *where* interference is happening at country scale, with a 6 to 24 hour refresh, only where aircraft fly, and they cannot tell jamming from spoofing. OpsGroup counted about 1,500 flights per day affected by spoofing in 2024.

What this means for TrustNav. These maps say "something is wrong in this region". TrustNav says "this van, since 16:08, 8 km off, authentication failed, hold its position". Different scale, different user.

7. Academic approaches

Recent papers use machine learning on receiver observables (C/N0, pseudorange residuals, Doppler, clock), mostly trained and tested on the TEXBAT recordings: for example an LSTM / VAE network (Tang and Qin, 2025, F1 96.9 % on TEXBAT), a convolutional VAE trained on clean data only (Iqbal et al., about 100 % true positives on easy sets, 89 % on the hardest), and a random forest on multi-parameter features (Li et al., IEEE Access 2025, above 97 %). A Stanford study (Kriezis et al., ION ITM 2025) ran rule-based checks on real spoofing in the eastern Mediterranean with a 200 EUR u-blox ZED-F9P: C/N0, residuals and clock drift were useful, AGC and Doppler were not, and the standard aviation MOPS algorithm scored 94 % on one receiver but under 13 % on another.

What this means for TrustNav. TrustNav is in the same family (rules plus a model trained on normal data only, on receiver observables) with three differences: it is a *system*, not a classifier, with incidents, explanations and a response; it adds the *route* and *odometer* as evidence the receiver cannot provide; and it reports *per-second trust* rather than a binary verdict. The Stanford result is also a warning we take seriously: thresholds tuned on one receiver can fail on another, which is why the real-receiver phase starts by recording normal drives and re-calibrating.

Side by side

	Receiver flags (u-blox, Septentrio)	Fixed probes (GPSPatron)	Anti-jam hardware (infiniDome)	Fleet telematics (Geotab etc.)	OSNMA	Aviation maps	TrustNav
Protects	one receiver	a site or region	one receiver	the fleet screen	Galileo data	a region	**each moving platform**
Detects	jamming + spoofing	jamming + spoofing	jamming (mitigates)	relays receiver jamming flag	forged Galileo data	degraded accuracy	**jamming + spoofing + forgery**
Per-second score	no (state code)	probability	no	no	no	no (daily / 6 h)	**yes, 0 to 100**
Explains why	no	partly (charts)	no	no	n/a	no	**yes, in words with values**
Uses route / odometer	no	no	no	no	no	no	**yes**
Responds	no	alerts	yes (RF)	report	flag	no	**refuses the fix, estimates, recommends action**
Extra hardware	none	probe + 1 to 3 antennas	2+ antennas	none	OSNMA-capable receiver	none	**none**
Fleet view	no	network of sites	no	yes	no	yes	**yes**
Status	shipping	shipping	shipping	shipping	operational	live	**prototype; simulated data and replays of real recordings**

What TrustNav does not do (yet)

- It has not been tested **live on its own receiver**. The detection table comes from one simulator. It has been replayed on public recordings: a clean real drive, and real jamming and spoofing from Jammertest 2024 (two devices, two days), where the first version missed a spoofed phone until receiver time was checked against the host clock. The products above are tested at Jammertest every year, on many receivers. Phase 2 (see ROADMAP.md (section 8)) is the live test.
- It cannot **remove** a jammer or a spoofer from the air. Only antenna hardware mitigates at RF level.
- It does not do **signal-level** analysis (spectrum, correlator shapes, antenna arrays). It works at the level of what a receiver reports, so a very careful spoofer that keeps every observable consistent and follows the road is hard to catch; the model layer and the odometer comparison are the defence, and a second antenna would be the next step.
- Its OSNMA status is **simulated**; a real implementation needs raw Galileo pages and an OSNMA library.

Sources

- u-blox: ZED-F9P HPG 1.50 release notes (https://content.u-blox.com/sites/default/files/documents/ZED-F9P_FW100HPG150_RN_UBXDOC-963802114-12826.pdf); NEO-M9N product summary; Jammertest 2025 blog (https://www.u-blox.com/en/blogs/tech/gnss-jamming-spoofing-detection-jammertest-2025-andoya); GPSPatron u-blox test (https://gpspatron.com/ublox-m8t-gps-spoofing-test); gnss.store prices.
- Septentrio AIM+: https://www.septentrio.com/en/learn-more/advanced-positioning-technology/aim-spoofing-protection and the Jammertest 2025 results page.

- GPSPatron: https://gpspatron.com/detecting-gnss-threats/ , https://gpspatron.com/gp-cloud , GP-Probe TGE2 datasheet.
- infiniDome: Honeywell GPSdome page; GPS World on GPSdome 2 (https://www.gpsworld.com/infinidome-releases-gpsdome-2-dual-band-anti-jamming-technology/).
- Regulus Cyber: Inside GNSS (https://insidegnss.com/?p=181662), GPS World, CB Insights, regulus.com.
- Safran BroadShield: https://safran-navigation-timing.com/product/broadshield
- Galileo OSNMA: EUSPA (https://www.euspa.europa.eu/newsroom-events/news/testing-operations-galileo-osnma-service-now-available-users), GSC FAQ (https://www.gsc-europa.eu/node/7832), Wang et al. 2025 (https://arxiv.org/html/2501.09246v1).
- Geotab: https://www.geotab.com/blog/combating-gps-jammers/ ; Navixy: https://navixy.com/docs/user/guide/events-and-notifications/device-positioning/gps-jamming
- Aviation maps: https://gpsjam.org/faq/ , https://www.flightradar24.com/data/gps-jamming , OpsGroup report (https://ops.group/blog/gps-spoofing-final-report/).
- Papers: Tang and Qin, APSIPA ASC 2025; Kriezis et al., ION ITM 2025 (https://web.stanford.edu/group/scpnt/gpslab/pubs/papers/Kreizis_ION_ITM_2025_Real_World_Spoofing_Detection.pdf); Li et al., IEEE Access 2025, doi 10.1109/ACCESS.2025.3553475; Iqbal, Aman, Sikdar (https://www.ece.nus.edu.sg/stfpage/bsikdar/papers/ccnc_25b.pdf); Bhuiyan et al., GPS Solutions 2026 (https://doi.org/10.1007/s10291-026-02102-z).
- Datasets: TEXBAT (https://radionavlab.ae.utexas.edu/texbat), Jammertest plan (https://github.com/NPRA/jammertest-plan), FGI Jammertest 2023 recordings (https://etsin.fairdata.fi/dataset/2eee8cfd-db00-4e46-bd62-d7632c09c700), VAARA, University of Vaasa, Jammertest 2024 (https://github.com/UniVaasaDigiEco/VAARA), Google gps-measurement-tools (https://github.com/google/gps-measurement-tools).
- Incidents: Inside GNSS on the Newark GBAS case; ANFR interference guide 2024 (https://anfr.fr/fileadmin/mediatheque/documents/brouillage/ANFR-DGITM_-*Guide_Brouillage_VD*-_2024.pdf); trans.info on the Lyon airport case; EU Council note ST-9188-2025 on Baltic interference.

8 Roadmap: prevention and improvements

The tutor's questions: *can it be improved more?* and *does it prevent attacks or only detect them?*

Prevention, honestly. No software can stop a jammer or a spoofer from transmitting; only antenna hardware mitigates at the radio level. What software *can* prevent is the damage: a fleet system acting on a false position. Since this version (engine 2.1) TrustNav does that with a **response layer** (`engine/trustnav_engine/safeguard.py`): a fix below the trust line is refused, the position handed downstream is estimated from the odometer along the planned route, and the operator receives an action (continue, verify, isolate). Measured on the held-out attacks, the worst position an ordinary screen would show was 8.0 km off; what the response layer passed on was never more than 29 m off, and during the 100 s jamming outage the screen never went blank. That is the prevention story today. Everything below makes it stronger.

Phase 2: real receiver (next two weeks)

#	Step	Why	Effort
0	**Use a phone as the receiver** (done, v10). The `/phone` page sends the phone's fixes; `python -m trustnav_engine.phone` scores them live; the page can overlay a teleport, a drift or jamming on the real stream.	Real GNSS data through the whole chain today, for free; the overlay shows detection and the response on real data without transmitting anything. The phone has no per-satellite C/N0, clock or OSNMA, so the physics and route rules carry the demo.	done
0b	**Replay a public real-receiver recording** (`python -m trustnav_engine.realcheck`, done): a 27.6-minute u-blox ZED-F9K drive with a SPAN reference. Rules: no false alarm; simulator model: re-calibration needed and demonstrated; overlaid attacks caught in 0, 35 and 1 s. Found and fixed two engine weaknesses (tail-score saturation, unbounded long-clean release).	Real receiver noise and real normal driving without buying anything yet.	done
0c	**Replay public recordings of real attacks** (`python -m trustnav_engine.attackcheck`, done, engine 2.2): Jammertest 2024, a phone captured by a real spoofer and a u-blox receiver under real jamming. The first version missed the capture for 21 min 49 s; a sixteenth rule on receiver time now refuses it in the same second. Four more weaknesses found and fixed.	Real attacks at the receiver level, recorded legally by others. Two days of one campaign: evidence, not a statistic.	done
1	**Plug in a real receiver.** A u-blox USB receiver on the laptop, `python -m trustnav_engine.receiver --port ... --no-model`. The adapter already exists and is tested on synthetic NMEA/UBX.	First real data. Shows the whole chain live: antenna, engine, cloud, dashboard.	1 hour once the device arrives
2	**Record normal drives** with `--jsonl` (walk, tram, car around Angers).	The model's idea of "normal" comes from the simulator. The Stanford study (Kriezis et al., 2025) found thresholds tuned on one receiver failed on another. Real normal data is the only cure.	2 to 3 hours of recording
3	**Re-calibrate** the rules' thresholds and retrain the model on those recordings (`calibrate` script, to write). Report the before/after false alarm rate.	Turns the research prototype into something that is honest about a real device.	1 day
4	**Replay raw radio recordings of attacks** through the same pipeline: the FGI Jammertest 2023 recordings (CC BY 4.0) and TEXBAT (research use) are raw RF, so they need a software receiver (GNSS-SDR) or a receiver fed by an SDR replay to produce observables; the Mendeley receiver-observable dataset can be used directly.	More attack types and more receivers than the two devices replayed so far, still without transmitting anything.	2 to 4 days

Phase 3: stronger evidence

#	Step	What it catches
5	**Vehicle odometer / OBD-II speed** as the inertial input (`--imu`).	Better dead reckoning during holds; a spoofed speed that disagrees with the wheels.
6	**Second antenna** on the same vehicle (two cheap receivers 1 to 2 m apart). Two real antennas see slightly different satellites and ranges; a single spoofer makes both report the same position.	Synchronous spoofing that fools every single-antenna check. This is how GPSPatron's probe works.
7	**Real OSNMA** from raw Galileo pages (UBX-RXM-SFRBX) with an open OSNMA library.	Forged Galileo data on a real receiver, instead of a simulated flag.
8	**Map matching** with OpenStreetMap roads for vehicles without an assigned route.	Drift onto places where no road exists, for any vehicle, not only fleets with planned routes.
9	**Supervised model** once labelled real attack data exists, kept beside the unsupervised one.	Known attack signatures with higher recall; the unsupervised model stays for the unknown ones.

Phase 4: operations

#	Step
10	Alerts by e-mail / SMS / webhook when an incident opens or an action becomes ISOLATE.
11	Drone integration: feed PX4 / MAVLink telemetry in, and send a "hold / return home on inertial navigation" command out when the response layer says ISOLATE.
12	Fleet-level analytics: where and when holds happen on a map over weeks (a jammer parked near a depot shows up as a cluster).
13	Multi-tenant accounts and per-fleet keys, so several operators can share one deployment.
14	Edge deployment on a Raspberry Pi in the vehicle, sending only scored frames to the cloud.

Research questions worth a thesis chapter

- How much detection delay does route-constrained dead reckoning buy back against a slow drift spoofer, as a function of odometer accuracy?
- Can two cheap receivers on one vehicle match a single expensive multi-antenna probe for synchronous spoofing?
- How transferable are "normal" models between receivers of the same chip family (M9 to F9, F9 to X20)?

Not planned

- Transmitting any jamming or spoofing signal, even in a lab, without a shielded chamber and a licence. All attack testing uses simulation and recordings made by others.
- Safety-critical use (aviation, maritime). The trust score is a research indicator, not a certified integrity figure.

9 Safety, legal and ethics

Nothing is transmitted. Every jamming, spoofing and forgery attack we made exists only as software: the simulator produces the observations a receiver would report, and the recorded scenarios replay them. The real attacks the detector has been replayed on were transmitted at Jammertest 2024 in Norway by the organisers, with the authorities' permission, and recorded by the University of Vaasa; we only read the logs they published. Transmitting on the GNSS bands is illegal in France (the ANFR investigates interference cases and the law provides for up to six months' imprisonment and a 30,000 EUR fine), and a jammer in a car affects everyone around it, as the Nantes and Lyon airport cases showed. Real attack testing uses public recordings (Jammertest 2024 so far; the raw captures of Jammertest 2023 and TEXBAT next), never the open air.

The receiver only listens. The Phase 2 hardware is a receive-only GNSS antenna and receiver, which needs no licence and emits nothing.

Personal data. The demo tracks seven simulated platforms; no personal data is involved. A real deployment tracks vehicles, and therefore the people driving them, so the GDPR applies: the purpose is to detect attacks on the positioning, not to monitor drivers; frames should be kept only as long as needed (the demo database prunes live frames after three days); access is controlled (public read of synthetic data only, writes through a token, presenter actions behind a passcode); and operators should tell drivers what is collected and why.

Security of the system itself. Secrets are never in the repository; the ingest token is stored only as a SHA-256 hash and the presenter passcode as a bcrypt hash with rate limiting; row-level security limits the browser to reading. The response layer is designed so that a spoofer cannot end a hold simply by looking clean (the returning fix must rejoin the estimate).

Honest claims. The trust score is a research indicator, not a certified integrity figure. The detection table comes from one simulator; the real attacks are two days of one campaign on two devices, and we publish that the first version missed the real spoofing. TrustNav is not for aviation, maritime or other safety-critical use. Its limits are listed in the Method page of the console and in the Results section.

Open data and attribution. Map data © OpenStreetMap contributors, basemaps © CARTO, road routes computed with OSRM. Real drive: Google gps-measurement-tools (Apache-2.0). Real attacks: VAARA dataset, University of Vaasa (Jammertest 2024); its repository states no licence, so its files are downloaded on demand and never redistributed here. Sources for the market comparison are cited in that section.

10 Team and contributions

Group 2, MSc Cybersecurity and Data Science, ESAIP Angers. Tutor: Sofiane Hamrioui.

Member	Role	Responsible for
Karan Deshmukh	Project lead	Overall design and architecture. The detection engine (simulator, features, rules, anomaly model, fusion, response layer, receiver adapter). The cloud side (database schema, row-level security, ingest function, demo feed). The console (map, replay, story, space-time view, presentation mode). The documentation and the communication with the tutor and the jury.
Niranjan Satheesh Thandassery	Evaluation and validation	The evaluation protocol (train, validation and held-out splits), the recorded attack scenarios and the results tables. Testing of the console and the end-to-end checks. The Phase 2 hardware plan: receiver and antenna choice, recording of normal drives, calibration.
Sudheer Reddy Jogi	Research and comparison	The market and literature study behind the comparison with existing solutions (receiver flags, fixed probes, anti-jamming hardware, fleet telematics, OSNMA, recent papers). Review of the documentation. Preparation of the oral presentation.
Venkata Pavan Kumar Sanam	Testing and presentation support	Functional testing of the live demo and the recorded scenarios, with issue reports. The safety, legal and ethics notes. The demo rehearsal and the presentation material for the oral examination.

How the team works. One repository on GitHub with every change reviewed by the lead; the live demo is redeployed automatically on every change; three tutor meetings (the first in the last week of October, the second and third before the oral examination on 13 November 2026); the first real receiver is demonstrated at the third meeting.

A Appendix: running the project

Run the engine

```
cd engine
python -m pip install -r requirements.txt
python -m pytest -q                      # 42 tests
python -m trustnav_engine.evaluate       # train, choose fusion weights, print held-out metrics
python -m trustnav_engine.export         # write out/*.json for the dashboard
python -m trustnav_engine.stream --dry-run --speed 10   # live engine without sending
python -m trustnav_engine.receiver --port /dev/ttyACM0 --no-model --dry-run   # a real u-blox receiver
(NMEA/UBX over USB)
python -m trustnav_engine.realcheck      # replay the public real-receiver recording, re-calibrate, overlay
attacks
                                         # (add --out ../paper/figures/realcheck.json
../dashboard/public/data/realcheck.json to refresh the paper and the site)
python -m trustnav_engine.attackcheck    # replay the public recordings of real jamming and spoofing
(downloads about 50 MB once)
                                         # (add --out ../paper/figures/attackcheck.json
../dashboard/public/data/attackcheck.json)
```

To stream live to the dashboard, put the ingest URL and token in `engine/.env.local` (never committed) and run:

```
set -a; source .env.local; set +a
python -m trustnav_engine.stream
```

Run the dashboard

```
cd dashboard
npm install
npm run dev
```

Space-time view (toggle at the top of the map, in Live and Replay): the ground is the map in metres and height is time, so each platform's track rises as a ribbon coloured by trust. A spoof tears the ribbon away from the real path; drift is a lean; jamming is a gap. Drag to rotate, wheel to zoom. Pure canvas, no tiles.

Opening screen offers a narrated 60-second story of an attack (also `#replay?scenario=teleport_spoof&story=1`, or "Watch the attack" in the search). The map is tilted with 3D buildings; `T` opens the fleet timeline dock.

Live console (v6): a fleet KPI strip (fleet trust, platforms, open incidents, positions refused and the worst lie refused); markers with a trust ring and score; a refused fix drawn hollow with a shield at the protected position, the dashed lie between them and its length; alert cards with sparklines, the response action and an acknowledge button; an operator playbook (what to do now) for any platform on hold; and **presentation mode** (`P`), where the camera visits every platform, incidents first, with a caption, for a jury or a wall screen.

On recordings of real attacks (v12): the Results page has a section that leaves the simulator: a phone captured by a real spoofer for 26.5 minutes (the reported error against the error handed on, the four detectors compared), a u-blox receiver under real jamming and beside the spoofer, and what the recordings changed in the engine. The earlier real-receiver section is now **On a real drive with no attack**. The Method page lists the sixteenth rule.

Opening scene, scoreboard and fusion lab (v11): the opening screen now plays the recorded teleport attack as a live scene (the van on its real Angers route, the reported position thrown 8 km away, the trust gauge falling from 95 to 12 and the protected position carrying on); the Results page opens with a scoreboard (precision, recall, false alarms, attacks caught, and bars comparing rules, model and combined); the Results page also has a section on a real receiver (the track of the public u-blox drive, false alarms before and after re-calibration, the three overlaid attacks); the Method page has a fusion lab where two sliders show what the noisy-OR fusion decides and what an average would have missed; and a link to the site now unfurls with a preview image of the opening scene.

Your phone as a live receiver (v10): open trustnav.vercel.app/phone on any phone, paste the ingest token and tap *Start sending*: the phone's own GNSS fixes go to the cloud once a second, `python -m trustnav_engine.phone` on a laptop scores them with the same rules and response layer as every receiver, and the live console shows the phone moving over the real streets of Angers with a trust score. A switch on the phone page overlays a teleport, a slow drift or jamming on the real stream (in software, nothing is transmitted), so detection and the response can be shown on real data without buying hardware. A phone gives position, speed and heading but no per-satellite signal strength, clock or OSNMA, which is what the dual-band receiver board adds later.

Opening screen in ten languages and voice everywhere (v9): the opening screen has a language menu (shared with the explainer); the 60-second attack story and presentation mode read their captions aloud, with a mute button on the caption.

Narrated explainer (v8): "Explain it to me, with voice" on the opening screen (also in the `?` guide and the ⌘K search) plays a three-minute, video-style walkthrough inside the console: nine scenes that move the camera, highlight one panel at a time, show a caption and speak it, in **ten languages** (English, French, German, Spanish, Italian, Portuguese, Hindi, Marathi, Telugu, Tamil). The voice is the browser's own speech synthesis, so it works offline; when the device has no voice for a language, the captions are timed instead (Marathi borrows the Hindi voice). Space pauses, the arrow keys step, Esc leaves.

Plain words (v7): every platform panel opens with a verdict a non-expert understands ("Satellite signal is being faked: the receiver places the van 8.0 km from the road it is on; TrustNav is tracking it by its odometer"), derived from the evidence only, so it works on a real receiver. Evidence lines use everyday wording with the engineering term in brackets. `?` opens **How to read this screen**: the map legend, the panels, and the three things that go wrong, in one page.

Keyboard: `1`–`4` switch pages, `?` opens the guide, `⌘K` / `Ctrl K` opens the search, `J` / `K` step through platforms, `Esc` goes back, `F` follows the selected platform, `R` opens its incident report, `T` the fleet timeline, `P` presentation mode, `D` toggles dark mode, `Space` and the arrow keys drive the replay.

End-to-end checks (every view, interactions, shortcuts, dark mode, phone layout, axe accessibility scan, JS budget):

```
npm run build && npx next start -p 3100 &
python3 tests/e2e.py http://localhost:3100
```

The dashboard reads public, synthetic data only. Writes go through the ingest function (token, stored as a SHA-256 hash) or the presenter controls (passcode, stored as a bcrypt hash, rate-limited).

Data sources

- Road geometry: OpenStreetMap contributors, routed with OSRM.
- Basemap: CARTO Dark Matter (OpenStreetMap data).
- Galileo OSNMA operational since 24 July 2025 (EUSPA).
- Real drive: Google gps-measurement-tools example recordings (Apache-2.0); a copy of the two files used is in `engine/trustnav_engine/data/real/`.
- Real attacks: VAARA, University of Vaasa, Jammertest 2024 (github.com/UniVaasaDigiEco/VAARA). The repository states no licence, so its files are never copied here: `attackcheck` downloads them on demand.

Roadmap

1. Real receiver: u-blox ZED-F9P with OSNMA firmware, Raspberry Pi, walk and tram tests in Angers.
2. Drone: PX4 simulator, then a real airframe.
3. Raw radio recordings of real attacks (Jammertest 2023, TEXBAT) through a software receiver. The receiver-level recordings of Jammertest 2024 are already replayed, see *Real attacks* above.

Links

- Live demo: https://trustnav.vercel.app

- Source code and documentation: https://github.com/domail2karandeshmukh-cyber/trustnav
- This document is built from the `docs/` folder of the repository with `python3 docs/build_pdf.py`.

Schedule

Three tutor meetings; oral examination on 13 November 2026, 09:35, room C201. Team and contributions are in section 10.